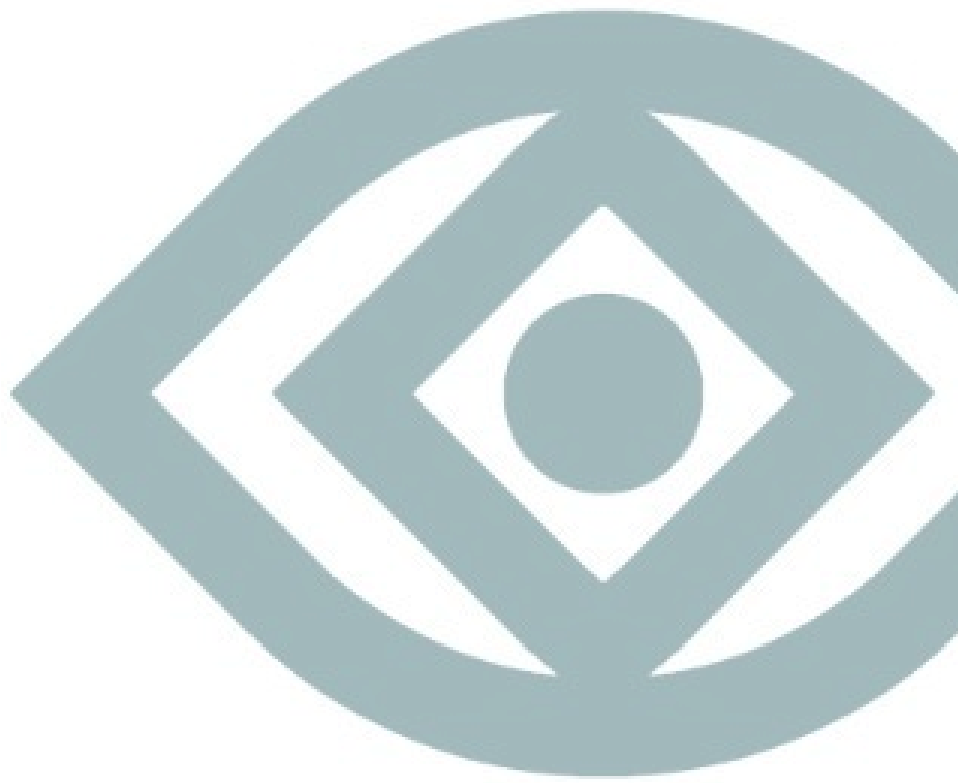


Plether Perpetuals

Pre Audit Report

4th May 2026



Prepared For:

Plether

<https://plether.com/>

Prepared By:

Linus L. (oot2k)

Ihtisham U. (ihtishamSudo)

SC Audit Studio OY

<https://scauditstudio.com/>

info@scauditstudio.com

Purpose of this Pre Audit

The purpose of a pre audit is to provide security focused protocol development and design consultation. The primary goal is to challenge the design, cleanup the code, check tests, and make sure the overall protocol has no major obvious vulnerabilities before the formal audit is done. This process shifts the focus of the final audit from finding everything to finding only deep security vulnerabilities.

For this reason codebases can not be considered production ready after a pre audit.

Vulnerability Classifications

High	A vulnerability leading to critical damage without constraints
Medium	A vulnerability leading to critical damage with constraints
Low	A vulnerability leading to little damage with constraints
Info	A vulnerability leading to no damage
Q/A	A code quality improvement recommendation

About SC Audit Studio

SC Audit Studio is a cyber security company providing consultation and audit services. Additionally we develop cyber security tools. Our mission is to create a more transparent and secure environment for everyone in a fast evolving world. We believe founders, innovators, and visionaries should be able to focus on building the future while we handle security.

Disclaimer

SC Audit Studio makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security review does not endorse the underlying business or product. This review was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

Project Details

Project	Plether Perpetuals
Website	https://plether.com/
Language	Solidity
Network	Arbitrum
Github Repository	https://github.com/Plether-Fi/plether-core/tree/master
Commit	fc5f03ba8b91e879e0fa7f0d9be208d4c518a305
Resolution	86263b00b6f92301c4cb62fa0aab4317f0cbe7cd

Executive Summary

Between April 14, 2026 and May 4, 2026, Plether engaged SC Audit Studio to perform a pre-audit consultation of the Plether Perpetuals protocol. During the engagement, two security researchers reviewed the system architecture, reviewed the smart contract design, identified potential vulnerabilities, and provided recommendations focused on security best practices and code quality.

Plether Perpetuals is a perpetual trading protocol designed around a forex-dominated index that tracks the value of the U.S. Dollar against a basket of other currencies. The protocol architecture is composed of four core contracts: the [OrderRouter](#), [ClearingHouse](#), [CfdEngine](#), and [HousePool](#), supported by several libraries and the vault contract.

The [OrderRouter](#) serves as the primary entry point for user interactions. It enables users to submit orders, allows keepers to execute pending orders, and permits liquidators to close unhealthy positions. During the review, recommendations were made regarding inheritance structure, security hardening, and overall code quality improvements.

The [ClearingHouse](#) is responsible for trade settlement and custody of user margin. It also manages keeper incentives, including bounty payouts for order execution and liquidations. During the review, recommendations were made to rework the bounty logic into a simpler design.

The [CfdEngine](#) contains the core accounting and trading logic of the protocol, including fee calculations, liquidation mechanics, and pricing validations. It is primarily invoked by the [OrderRouter](#) and synchronizes the accounting state with the [ClearingHouse](#). Prior to trade execution, the engine performs critical security checks and enforces pricing constraints.

The [HousePool](#) acts as the protocol's shared counterparty liquidity pool. It manages liquidity provider funds across senior and junior tranches through the [TrancheVault](#) system, which implements time-locked withdrawals. The senior tranche is designed to generate relatively stable yield, while the junior tranche absorbs first losses in exchange for higher potential returns. Recommendations were made to further align the vault architecture with ERC-4626 design principles to improve standardization and interoperability.

Overall, the engagement highlighted the importance of conducting a pre-audit review for complex financial protocols. Given the natural complexity of perpetual trading systems, establishing strong security practices and best design principles early in development is essential to reducing risk and improving maintainability.

Total Findings

High	Medium	Low	Info
2	7	1	18

Code Quality Improvements

SC-Q1: Naming should be consistent across the codebase

Summary

Three distinct naming inconsistencies pollute the codebase and create an understatement gap between what concepts are called in different files, comments, and interfaces:

1. **HousePool is referred to as vault throughout**, `ICfdVault`, `ICfdEngineCore.vault`, `OrderRouter/OrderExecutionOrchestrator`
2. **Modules are both external contracts and inherited abstracts simultaneously** - `OrderExecutionOrchestrator` and `OrderOracleExecution` are named "modules" yet are abstract base contracts that `OrderRouter` inherits. `CfdEngineSettlementModule` and `CfdEngineAdmin` are deployed as independent contracts..
3. **Funding is called "carry" in some places and "funding" in others** - `CfdEngine` emits `CarryCheckpointed` / `CarryRealized`, stores `unsettledCarryUsdc`, and uses `baseCarryBps` in `RiskParams`. But the same concept appears as "funding" in error names (`CfdEngine__FundingExceedsMargin`, `CfdEngine__PartialCloseUnderwaterFunding`)

Code Snippet

```
None
// HousePool referred to as "vault" at the storage and interface
level

ICfdVault public vault;                                //
CfdEngine.sol, should be pool/housePool

interface ICfdVault { ... }                             // name implies
ERC-4626 vault, not a tranchd pool

vault.totalAssets()                                    // OrderRouter,
same object, different name
```

```

// "Module" means both inherited abstract and deployed contract

abstract contract OrderExecutionOrchestrator    // inherited
"module",  not deployed

contract CfdEngineSettlementModule              // deployed
"module",  external contract

// Carry vs Funding split across the same contract

event CarryCheckpointed(...)                    // CfdEngine,
uses "carry"

event CarryRealized(...)                        // CfdEngine,
uses "carry"

error CfdEngine__FundingExceedsMargin()         // CfdEngine,
uses "funding"

error CfdEngine__PartialCloseUnderwaterFunding() // CfdEngine,
uses "funding"

```

Recommended Fix

- Rename `ICfdVault` → `IHousePool` and rename the `vault` field on `CfdEngine` to `pool`.
- Split the module naming convention.
- Standardize on "carry" everywhere.

Response

Implemented in [Plether-Fi/plether-core#10](#)

SC-Q2: Move Oracle Logic from `OrderRouter` to a Dedicated Oracle Contract

Summary

The `OrderRouter.sol` contract currently contains oracle update logic and additional pricing logic implemented through a price library. These responsibilities should be removed from `OrderRouter` and transferred to a dedicated `PletherOracle` contract. This can be achieved by creating a new contract with the following functions:

- `updatePrice(refundRecipient, updateData)`: This function should perform all required Pyth update calls, update the oracle price, apply configured price caps/guards, and finally refund any remaining ETH balance to `refundRecipient`. Centralizing refund handling here removes the need for nested `sendETH` calls in the router flow.
- `getLatestPrice()`: This view function should return the latest basket price, so it can be reused cleanly in several places in `OrderRouter`.

Code Snippet

None

```
interface IPletherOracle {
    function updatePrice(
        address refundRecipient,
        bytes[] calldata updateData
    ) external payable returns (uint256 latestPrice);
    function getLatestPrice() external view returns (uint256
latestPrice);
}
```

Recommended Fix

- Move all oracle/Pyth update logic from `OrderRouter` into `PletherOracle`.
- Keep `OrderRouter` focused on order execution only.
- Adjust naming for clarity and consistency (for example, align terms like `latestPrice`, `basketPrice`, and `markPrice` across contracts).

Response

Implemented in: [Plether-Fi/plether-core#15](#)

SC-Q3: Shared state should be defined in parent of the inheritance tree

Summary

`OrderRouter` collapses distinct failure modes into generic numeric-code errors (e.g., `OrderRouter__CommitValidation(uint8 code)`) using private wrappers like `_revertCommitValidation`. This obfuscates on-chain readability and makes ABI decoding fragile by hiding semantic violations behind opaque integer lookup tables.

Configuration fields accessed in parent abstract contracts (`OrderExecutionOrchestrator`, `OrderOracleExecution`) currently reside in `OrderRouter`, requiring redundant getter overrides like `_queueHeadOrderId` and `_maxOrderAge`. Moving these fields to the abstract contracts that utilize them eliminates this boilerplate.

Code Snippet

Opaque error codes, all distinct failure modes collapsed into three error types:

```
None
// OrderRouter.sol

error OrderRouter__OracleValidation(uint8 code);

error OrderRouter__QueueState(uint8 code);

error OrderRouter__CommitValidation(uint8 code);

// 13+ oracle codes routed through a single wrapper

function _revertOracleValidation(uint8 code) internal pure {

    revert OrderRouter__OracleValidation(code);

}

function _revertEmptyFeeds()                internal pure override
{ _revertOracleValidation(0); }

function _revertLengthMismatch()            internal pure override
{ _revertOracleValidation(1); }
```

State defined in the child, accessed in the parent via mandatory getter overrides:

```
None
// OrderRouter.sol, state owned here ...

uint64 public nextCommitId = 1;

uint64 public nextExecuteId = 1;

uint64 public globalTailOrderId;

uint256 public maxOrderAge;

uint256 public maxPendingOrders;
```



```
// ... but consumed in OrderExecutionOrchestrator via these
overrides:

function _queueHeadOrderId() internal view override returns
(uint64) { return nextExecuteId;      }

function _setQueueHeadOrderId(uint64 id) internal override
{ nextExecuteId = id;                }

function _queueTailOrderId() internal view override returns
(uint64) { return globalTailOrderId; }

function _setQueueTailOrderId(uint64 id) internal override
{ globalTailOrderId = id;            }

function _maxOrderAge() internal view override returns
(uint256) { return maxOrderAge;      }

function _nextCommitId() internal view override returns
(uint64) { return nextCommitId;      }
```

Recommended Fix

1. Replace numeric-code error families with one named error per failure mode.

Each `_revertXxx()` override should become a concrete named error declared in the relevant abstract contract (or a shared `IOrderRouterErrors` interface). This makes every revert self-documenting, eliminates the code dispatch wrappers, and allows callers to match on a specific selector rather than decoding a generic `(uint8)` payload.

For `CommitValidation` codes 0–10, the same applies, each maps to a distinct semantic condition (`NotInSeedLifecycle`, `VaultRiskBlocked`, `CloseWithPositiveMargin`, `NoQueuedPosition`, `SideMismatch`, `SizeExceedsQueued`, `InsufficientFreeEquity`, `TooManyPendingOrders`, `Unauthorized`, `DegradedMode`, `CloseOnlyWindow`) and deserves its own error type.

2. Move the shared queue and config state to the lowest abstract contract that consumes it.

Since `nextExecuteId`, `globalTailOrderId`, `maxOrderAge`, `nextCommitId`, and `maxPendingOrders` are exclusively accessed via abstract base methods, declaring them directly in `OrderExecutionOrchestrator` (or a dedicated `OrderRouterState` base) removes the entire getter/setter override layer:

None

```
// In OrderExecutionOrchestrator, owns the state it manages
uint64 internal _nextExecuteId = 1;
uint64 internal _globalTailOrderId;
uint256 internal _maxOrderAge;
// No overrides needed, base methods read the fields directly
function _queueHeadOrderId() internal view returns (uint64) {
    return _nextExecuteId; }
```

`OrderRouter` then only declares state that is genuinely its own responsibility (`nextCommitId`, `admin`, bounty configuration), and the six boilerplate getter overrides disappear completely.

Response

Implemented in [Plether-Fi/plether-core#18](#)

SC-Q4: Rework House Pool

Summary

The current `HousePool` / tranche system double-accounts deposits: it uses an ERC-4626 vault architecture but does not leverage the vaults' native asset and share accounting. Additionally, it promises a Fixed senior yield while the underlying yield source is unpredictable trading activity, which creates the edge cases reported in a separate issue.

We propose a reworked pool design that preserves the overall feature set while significantly reducing code complexity.

Code Snippets

The following can be deleted under the proposed design:

HousePool.sol

- Imports: lines `HousePool.sol`, all `HousePool*Lib` imports become unnecessary
- State variables: lines `HousePool.sol`, `seniorPrincipal`, `juniorPrincipal`, `unpaidSeniorYield`, `seniorHighWaterMark`, `accountedAssets`, `unassignedAssets`, `pendingRecapitalizationUsdc`, `pendingTradingRevenueUsdc`
- `depositSenior` / `withdrawSenior`
- `depositJunior` / `withdrawJunior`
- `reconcile` (public) + `_reconcile` (internal)
- `assignUnassignedAssets` / `initializeSeedPosition`
- `accountExcess` / `sweepExcess`
- Waterfall helpers: `_getWaterfallState`, `_setWaterfallState`, `_distributeRevenue`, `_absorbLoss`: lines `HousePool.sol`

- Pending-claimant bucket helpers: `_applyPendingClaimantBucketsLive`, `_applyPendingClaimantBucketsPreview`, `_getPendingClaimantBuckets`, `_clearPendingClaimantBuckets`, `_recordPendingClaimantInflow`, `_copyPendingAccountingState`, `_pendingClaimantBucketAssets`
- Reconcile preview helpers: `_previewPendingAccountingState`, `_checkpointSeniorYieldBeforePrincipalMutation`, `_normalizeUnassignedAssets`, `_requireNoPendingBootstrap`

Library files - fully removable:

- `HousePoolAccountingLib.sol`
- `HousePoolPendingLivePlanLib.sol`
- `HousePoolPendingPreviewLib.sol`
- `HousePoolReconcilePlanLib.sol`
- `HousePoolSeedLifecycleLib.sol`
- `HousePoolTrancheGateLib.sol`
- `HousePoolWaterfallAccountingLib.sol`
- `HousePoolWithdrawalPreviewLib.sol`

Recommended Fix

Implement a new pool structure that follows the ERC-4626 vault design. In this system the `HousePool` contract is responsible only for security checks and PnL settlement, it neither holds funds nor tracks asset balances. The senior tranche acts as the main entry point for user deposits, while the junior tranche allows users to "re-stake" senior shares by using them as its underlying asset.

Response

Acknowledged

SC-Q5: Move Custom Errors and Events into Interfaces

Summary

`HousePool`, `CfdEngine` and `OrderRouter` each define their custom errors, events, and several internal structs **directly inside the contract body** rather than inside the interfaces those contracts implement. This breaks the ABI-source-of-truth principle: external consumers, test harnesses, and protocol integrators that depend on the interface layer must import the concrete contract just to reference these definitions, creating tight coupling and undermining the purpose of the interface layer entirely.

The pattern is especially painful for custom errors and events because test suites already reference them via the interface (`ICfdEngine.CfdEngine__TypedOrderFailure`, etc.) in some places, but the canonical declarations live in the implementation, forcing redundant dual-declarations or awkward cross-imports.

Code Snippet

CfdEngine.sol, errors and events scattered in the implementation body:

```
None
// src/perps/CfdEngine.sol (lines ~187-253)

error CfdEngine__Unauthorized();

error CfdEngine__VaultAlreadySet();

// ... ~28 more errors ...

event FundingUpdated(int256 bullIndex, int256 bearIndex, uint256
absSkewUsdc);

event PositionOpened(...);

// ... ~13 more events ...
```

ICfdEngine.sol, forced re-declaration of two errors to allow interface-side **abi.encodeWithSelector** in tests:

```
None
// src/perps/interfaces/ICfdEngine.sol (lines 17-19)

error CfdEngine__TypedOrderFailure(
    CfdEnginePlanTypes.ExecutionFailurePolicyCategory
failureCategory, uint8 failureCode, bool isClose
);

error CfdEngine__MarkPriceOutOfOrder();
```

HousePool.sol, all errors and events in the contract, **IHousePool** declares none:

```
None
// src/perps/HousePool.sol (lines ~88-130)

error HousePool__NotAVault();

error HousePool__SeniorImpaired();
```

```
// ...

event Reconciled(uint256 seniorPrincipal, uint256
juniorPrincipal, int256 delta);

event TradingActivated();

// ...
```

OrderRouter.sol, errors and the single event defined after state variables:

```
None
// src/perps/OrderRouter.sol (lines ~44–51)

error OrderRouter__ZeroSize();

error OrderRouter__OracleValidation(uint8 code);

error OrderRouter__CommitValidation(uint8 code);

error OrderRouter__InsufficientGas();

error OrderRouter__PredictableOpenInvalid(uint8 code);

event OrderCommitted(uint64 indexed orderId, bytes32 indexed
accountId, CfdTypes.Side side);
```

Recommended Fix

1. Move all custom errors, events, and externally-visible structs into the relevant interface.

Each contract already has a primary interface ([ICfdEngine](#), [IHousePool](#), and a forthcoming [IOrderRouter](#)). Errors, events, and structs that are externally observable (referenced by tests, integrators, or other contracts via [abi.encode/abi.decode](#)) should be the sole declaration in that interface. The contract then inherits or implements the interface, and the definitions flow down automatically, no duplication.

None

```
interface IHousePool {

    // — Errors
    error HousePool__NotAVault();

    error HousePool__SeniorImpaired();

    error HousePool__DegradedMode();

    // ... all HousePool__ errors ...

    // — Events
    event Reconciled(uint256 seniorPrincipal, uint256
juniorPrincipal, int256 delta);

    event TradingActivated();

    event SeniorRateProposed(uint256 newRateBps, uint256
activationTime);

    // ... all HousePool events ...

    // — View structs
    struct VaultLiquidityView { ... }

    // ... all view-facing structs ...

    // — Function signatures
    function depositSenior(uint256 amount) external;

    // ...
}
// src/perps/HousePool.sol
contract HousePool is IHousePool, ICfdVault, IPerpsLPActions,
Ownable2Step, Pausable {

    // No error or event declarations, inherited from IHousePool

    ...
}
```

2. Eliminate the duplicated error declarations in `ICfdEngine` / `ICfdEngineCore`.

Once all errors are moved to `ICfdEngine`, `ICfdEngineCore` can `import` and inherit `ICfdEngine` (or a shared errors-only interface) rather than re-declaring the same error selectors in two places.

3. For `OrderRouter`, create `IOrderRouter.sol` (currently missing as an interface for errors/events) and move `OrderRouter__*` errors and `OrderCommitted` there. This is consistent with the existing `IOrderRouterAccounting` / `IOrderRouterAdminHost` split but completes the surface.

4. Keep internal structs private. Structs like `HousePoolContext` and `PendingAccountingState` that are purely internal implementation details should remain in the contract or be extracted to a library. Only externally observable return-type structs (`VaultLiquidityView`, `LiquidationPreview`, `SideState`) need to live in interfaces.

This change has zero runtime cost, reduces test import coupling, and ensures that any downstream consumer can detect reverts and decode events with only the interface import, the correct Solidity architecture for auditable, upgradeable protocol contracts.

Response

Implemented in [Plether-Fi/plether-core#20](#)

SC-Q6: Internal getter function in `MarginClearinghouseAccountingLib.sol` should be removed

Summary

`MarginClearinghouseAccountingLib` defines four internal getter functions, `getSettlementBalanceUsdc`, `getFreeSettlementUsdc`, `getPositionMarginUsdc`, and `getQueuedReservedUsdc`, that do nothing beyond returning a named field from `IMarginClearinghouse.AccountUsdcBuckets`. Every call site that uses these getters could read the struct field directly with identical gas cost and superior readability. The wrappers add an unnecessary indirection layer with no abstraction value.

Code Snippet

None

```
// Each of these is a single-line field read dressed up as a function:
```

```
function getSettlementBalanceUsdc(
```

```
    IMarginClearinghouse.AccountUsdcBuckets memory buckets
```

```

) internal pure returns (uint256) {

    return buckets.settlementBalanceUsdc;        // identical to
    buckets.settlementBalanceUsdc

}

function getFreeSettlementUsdc(...) internal pure returns
(uint256) {

    return buckets.freeSettlementUsdc;

}

function getPositionMarginUsdc(...) internal pure returns
(uint256) {

    return buckets.activePositionMarginUsdc;    // note: getter
    name diverges from field name
    ...

```

The name divergence between `getPositionMarginUsdc` → `activePositionMarginUsdc` and `getQueuedReservedUsdc` → `otherLockedMarginUsdc` is a subtle additional hazard: a reader must trace through the getter to understand which field is actually being read.

Recommended Fix

Delete the four getter functions and replace all call sites with direct field access. If the field name `activePositionMarginUsdc` is considered too verbose, rename the struct field itself, a single change propagated by the compiler is safer than an alias function that can silently diverge.

```

None
// Before

uint256 freeSettlementUsdc = getFreeSettlementUsdc(buckets);

uint256 positionMarginUsdc = getPositionMarginUsdc(buckets);

...

```



```
// After

uint256 freeSettlementUsdc      = buckets.freeSettlementUsdc;

uint256 positionMarginUsdc     = buckets.activePositionMarginUsdc;

...
```

Response

Implemented in [Plether-Fi/plether-core#10](#)

SC-Q8: Improve folder and file structure by clear split into respected categories

Summary

Split the project into clear folders: Interfaces for only Interfaces, Types for all Types, libraries for all libraries. Create one interface for one contract, name the interface after the contract, only use the single interface across the codebase.

Code Snippets

Related to all contracts.

Recommended Fix

1. **Create `src/perps/types/`** and move all `*Types.sol` files there (`CfdTypes`, `CfdEnginePlanTypes`, and the seven `*ViewTypes` files currently sitting in `interfaces/`).
2. **Move all structs/enums out of interface files** into their corresponding types file. Interface files should contain only `function` signatures. Alternatively remove types completely
3. **One interface per contract.** Merge the fragmented `ICfdEngine*` files into a single `ICfdEngine.sol`, `IOrderRouter.sol`, `IPerpsTrader.sol`, and `IPerpsLP.sol`.
4. **Mirror this layout at the top level**, interfaces already exists but `src/types/` does not; apply the same split there for `ISyntheticSplitter`, `IMorpho`, etc.

Response

Acknowledged

SC-Q9: Rework Credit and Deferred Payout Logic for Bounties

Summary

The current bounty accounting logic is fragmented, resulting in duplicated state and tracking across `CfdEngine`, `OrderRouter`, and `OrderEscrowAccounting`. The deferred accounting path also unnecessarily forces keepers to underwrite risk. These issues can be resolved by viewing all fees as a combined **bounty concept**. Bounties should be re-worked as internal margin transfers within the **clearinghouse**, utilizing the distinction between "real assets" and "virtual assets," thus eliminating the need to interact with the **vault** and the deferred accounting path. Specifically, **Liquidation bounties** must be direct margin transfers and not touch the **vault**. This ensures bounties can always be paid out.

Constraints on withdrawability can be enforced at the user accounting level, e.g. if the user's account is not healthy, bounty credits cannot be withdrawn.

Liquidation bounties should not touch the vault. They should not be subject to overall protocol health, and should only mark a direct margin transfer from the liquidated account to the keeper's clearinghouse account (see GMX, Hyperliquid).

Code Snippets

Fragmented bounty functions in `CfdEngine`: `CfdEngine.sol`: Lines 442-500

These three separate entry-points - `absorbRouterCancellationFee`, `recordRouterProtocolFee`, and `creditKeeperExecutionBounty`, represent the same concept (routing value from one on-chain party to another)

Deferred keeper credit path triggered in `OrderRouter`: `OrderRouter.sol`: Lines 458-482

The `_creditOrDeferLiquidationBounty` function queries vault cash, computes a `CashPriorityLib.SeniorCashReservation`, conditionally calls `vault.payOut(...)`, and falls back to `engine.recordDeferredKeeperCredit(...)`. This means a liquidation bounty payment involves up to three contracts (vault, clearinghouse, engine)

Escrow collection path in `OrderEscrowAccounting`: `OrderEscrowAccounting.sol`: Lines 163-215

`_collectExecutionBounty` transfers USDC from the router to the clearinghouse before calling `engine.creditKeeperExecutionBounty(...)`. The USDC transfer is redundant: the bounty was already seized from the trader's clearinghouse account at commit time (`OrderEscrowAccounting.sol`: Lines 108-142) and custodied by the router. A direct clearinghouse-to-clearinghouse credit would be sufficient.

Close bounty reservation in `CfdEngine`: `CfdEngine.sol`: Lines 635-670

`reserveCloseOrderExecutionBounty` is a separate reservation path for close orders that differs from the open-order flow managed entirely inside `OrderEscrowAccounting`. This asymmetry means the two paths have divergent failure modes and margin accounting.

Deferred credit state tracking: CfdEngine.sol: Lines 601-640

`claimDeferredKeeperCredit` requires the vault to be liquid before settling to the clearinghouse, even though the credit is simply a balance increment inside the clearinghouse. The vault check is only necessary because the current architecture originates keeper value from the vault; if the source is always trader margin (a direct seize), this check becomes unnecessary.

Recommended Fix

The final claim structure should be:

- **Protocol fees and keeper bounties on open/close** should be simple margin transfers, accounted for only when a trader has margin, and require no vault interaction.
- **Liquidation bounties** should be treated as senior claims to ensure liquidations remain incentivized regardless of vault state. They are capped by the liquidated account's reachable collateral.
- **Protocol fees on liquidations** can be paid only if margin remains after the vault PnL and keeper bounty are accounted for.

Fee reservation at commit time: Fees (open/close bounties and protocol fees) must be reserved from trader margin at order creation (`commitOrder`) and held in the clearinghouse's custody.

Single `creditBounty()` in Engine: A single `creditBounty(bytes32 fromAccount, bytes32 toAccount, uint256 amountUsdc)` function should replace multiple existing functions and the deferred credit path. This function debits the source account via `clearinghouse.seizeUsdc` and credits the recipient via `clearinghouse.settleUsdc`, then emits a `BountyCredited` event. The source account varies based on the order type (liquidated trader or per-order escrow reservation).

Response

Acknowledged

SC-Q10: Rework Router Structure

Summary

The `OrderRouter` is the contract that houses the external entry points to open trades and liquidate positions. It inherits `OrderExecutionOrchestrator`, `OrderEscrowAccounting`, `OrderOracleExecution`, and `OrderQueueBook`.

The inheritance structure of the contract should be reworked to be split into logical parts (see GMX).

Code Snippets

The current inheritance chain is deep and difficult to follow:

OrderRouter → OrderExecutionOrchestrator → OrderOracleExecution →
OrderEscrowAccounting OrderQueueBook → OrderEscrowAccounting

Additionally scattered error routing (described in another report).

The `commitOrder` function performs validation, bounty quoting, margin reservation, and queue linking inline in a single ~80-line function with no clear separation of concerns.

- OrderRouter - Lines: 182-256

`executeLiquidation` calls four distinct internal helpers in sequence

- OrderRouter - Lines: 537-558

Mixed Responsibilities in Modules

- OrderExecutionOrchestrator - Lines: 65-300

Recommended Fix

The new inheritance structure can look similar to the following implementation:

OrderRouter

- ├─ OrderHandler (open/close order logic)
- ├─ LiquidationHandler (liquidation logic)
- ├─ BountyHandler (keeper and protocol fee accounting)
- └─ Validation (pre-execution checks)

OrderRouter

- Implements only external-facing functions (`commitOrder`, `executeOrder`, `executeOrderBatch`, `executeLiquidation`, `updateMarkPrice`).
- Inherits all handlers/modules listed below.
- Contains no business logic itself; each external function delegates immediately to the appropriate handler.

Modules / Handlers:

OrderHandler

- Responsible for opening and closing order logic.
- Includes margin reservation, queue linking, and engine calls for order execution.
- Absorbs the relevant parts of `OrderExecutionOrchestrator` and `OrderEscrowAccounting`.

LiquidationHandler

- Responsible for liquidation logic.
- Includes position liquidation engine call, and post-liquidation order clearing.
- Absorbs `_forfeitEscrowedOrderBountiesOnLiquidation`, `_clearLiquidatedAccountOrders`

BountyHandler

- Responsible for keeper, liquidation and protocol fee accounting.

- Encapsulates bounty logic.
- Consolidates the bounty-related helpers currently spread across [OrderRouter](#) and [OrderEscrowAccounting](#).

Validation

- Responsible for pre-execution validation of orders.
- Encapsulates pause checks, degraded-mode checks, close-only window checks, vault risk checks, and slippage checks.
- Removes the inline validation blocks from [commitOrder](#) and [executeOrder](#).

This way a clear execution pattern across all functions can be implemented. For example:

Commit Order in Router

1. [Validation](#) - pause state, degraded mode, close-only window, vault risk, zero size, predictable failure pre-filter.
2. [BountyHandler](#) - quote and reserve execution bounty.
3. [OrderHandler](#) - reserve margin, record order, link into queue.

Response

Implemented in [Plether-Fi/plether-core#9](#)

SC-Q11 Test Suite Should Follow Clear Structure

Summary

The src/perps test suite is already broad and security-aware. The main weakness is not missing test volume; it is organization, repeatability, and audit signal.

The suite should be restructured so an auditor can immediately distinguish:

product/spec behavior,

- accounting correctness,
- invariant/property testing,
- preview-vs-execution parity,
- historical regressions,
- gas profiling,
- fork/integration coverage.

Right now those concerns are mixed together, especially in test/perps, where core tests, invariant suites, gas tests, and many audit-history *Failing.t.sol files live side by side.

Suggested structure:

Suggested test categories:

- spec: direct intended behavior from README.md, ACCOUNTING_SPEC.md, SECURITY.md, and PRE_AUDIT_GUIDE.md.
- accounting: close, liquidation, carry, deferred credit, fee, bad debt, and HousePool accounting.
- matrices: exhaustive policy tables for oracle regimes, payout modes, order failures, and withdrawal gates.
- differential: preview/live parity, planner/apply parity, public lens/internal lens parity.
- invariant: stateful actor tests with ghost ledgers and independent accounting models.
- regression: past audit findings and bug repros only.
- gas: explicit gas benchmarks, not mixed into correctness tests.
- fork: Pyth/USDC/mainnet-shape integration checks.

Suggested profiles:

[profile.quick.fuzz] runs = 256

[profile.ci.fuzz] runs = 2000

[profile.audit.fuzz] runs = 20000

[profile.audit.invariant] runs = 256 depth = 1000

Recommended Fix

1. Move all *Failing.t.sol files into test/perps/regression/audit/ and make clear whether each file is expected to pass, expected to fail, or preserved only as historical context.
2. Add a test/perps/TEST_MAP.md that maps every critical invariant to the exact test files covering it.
3. Increase invariant depth. Current [invariant] runs = 16 is too low for pre-audit confidence.
4. Strengthen independent ghost accounting for:
 - a. router escrow,
 - b. committed margin,
 - c. deferred trader credit,
 - d. deferred keeper credit,
 - e. bad debt,
 - f. fee custody,
 - g. LP withdrawal reserves.
5. Add matrix coverage for:
 - a. live / stale / frozen / FAD / degraded oracle regimes,
 - b. open / close / liquidation execution outcomes,
 - c. successful / failed / expired / blocked order lifecycle,
 - d. immediate payout / deferred payout / bad debt transitions.
6. Make preview/live parity a first-class audit target for close, liquidation, withdrawal, deferred claims, and LP views.
7. Keep unit tests small and domain-specific. Large files like CfdEngine.t.sol, OrderRouter.t.sol, and HousePool.t.sol should be split by behavior so auditors can review one risk area at a time.

8. Remove all the audit related tests cased from the codebase as it's not belong to the codebase.

Findings

SC-H1: Deposit Before Yield is Accrued Allows to Dilute Yield.

Summary

The HousePool system accrues fixed senior yield, which is initially recorded as `unpaidSeniorYield`, and later when the real yield is paid, synced to senior vault's principal. This now allows to frontrun the actual yield payment, diluting the yield for previous depositors.

Impact

Yield can be diluted leading to loss of funds for previous depositors.

Recommended Fix

Rework the vault accounting system.

Response

Completed senior rewrite in [Plether-Fi/plether-core#23](#).

This change requires a second audit.

SC-H2: Zero-Amount Withdraw/Redeem Can Perpetually Reset TrancheVault Cooldown and Lock Funds.

Summary

Zero-amount calls to `TrancheVault.withdraw()` and `TrancheVault.redeem()` allow any external address to exploit `_withdraw()`'s cooldown reset mechanism. Because a zero-amount allowance check succeeds without approval, an attacker can repeatedly reset a victim LP's withdrawal cooldown at no cost, perpetually locking their funds by forcing `maxWithdraw` and `maxRedeem` to return 0.

In `src/perps/TrancheVault.sol`, `withdraw()` and `redeem()` route into `_withdraw()`:

None

```
function withdraw(uint256 assets, address receiver, address
_owner) public override returns (uint256) {
    POOL.reconcile();
```

```

    ...
    return super.withdraw(assets, receiver, _owner);
}
function redeem(uint256 shares, address receiver, address _owner)
public override returns (uint256) {
    POOL.reconcile();
    ...
    return super.redeem(shares, receiver, _owner);
}

```

OpenZeppelin ERC4626 only checks whether the requested amount exceeds the owner's max:

```

None
if (assets > maxAssets) {
    revert ERC4626ExceededMaxWithdraw(owner, assets, maxAssets);
}
if (shares > maxShares) {
    revert ERC4626ExceededMaxRedeem(owner, shares, maxShares);
}

```

For assets == 0, this check passes even if maxAssets == 0.

The same applies to redeem(0, ...):

Then TrancheVault._withdraw() performs the cooldown reset before allowance spending, burning, or pool withdrawal:

```

None
if (block.timestamp < lastDepositTime[_owner] + DEPOSIT_COOLDOWN)
{
    revert TrancheVault__DepositCooldown();
}
lastDepositTime[_owner] = block.timestamp;
if (caller != _owner) {
    _spendAllowance(_owner, caller, shares);
}
_burn(_owner, shares);

```

For a zero-amount withdrawal, shares == 0. OpenZeppelin ERC20 allowance spending does not require any existing allowance when the spend amount is zero:

```

None
if (currentAllowance < value) {

```



```
        revert ERC20InsufficientAllowance(spender, currentAllowance,
value);
    }
```

So an attacker can call:

```
juniorVault.withdraw(0, attacker, victim);
juniorVault.redeem(0, attacker, victim);
```

or the same on the senior vault.

Attack flow:

1. Victim deposits funds and becomes withdrawable after the `DEPOSIT_COOLDOWN` expires.
2. Attacker calls `vault.withdraw(0, attacker, victim)` or `vault.redeem(0, attacker, victim)`.
3. The zero-amount call succeeds because `_spendAllowance` allows a zero value spend without prior approval.
4. `_withdraw()` resets the victim's `lastDepositTime`, forcing `maxWithdraw` and `maxRedeem` to return 0 until the cooldown expires again.

This can be [repeated](#) each time the cooldown expires.

Impact

A victim LP's funds can be repeatedly denied withdrawal without the attacker owning shares, transferring assets, or receiving approval.

Recommended Fix

Reject zero-amount withdrawals and redeems before reaching `_withdraw()`:

```
None

function withdraw(uint256 assets, address receiver, address
owner) public override returns (uint256) {
    require(assets > 0, "ZERO_ASSETS");
    ...
}

function redeem(uint256 shares, address receiver, address owner)
public override returns (uint256) {
    require(shares > 0, "ZERO_SHARES");
}
```

Response

Fixed in [Plether-Fi/plether-core#21](#)

SC-M1: Frozen-Window Liquidations Can Revert on Live Basket Timestamp Divergence

Summary

During oracle-frozen windows, liquidation staleness is relaxed to `fadMaxStaleness`. However, multi-feed baskets incorrectly use the stricter live-market `liquidationStalenessLimit` for checking timestamp divergence, causing liquidations to revert even when individual Pyth feeds are fresh enough.

The issue stems from passing the live liquidation limit as the basket publish-time divergence limit in `src/perps/modules/OrderOracleExecution.sol`:

```
None
maxStaleness = OracleFreshnessPolicyLib.getPolicy(
    OracleFreshnessPolicyLib.Mode.Liquidation,
    _isOracleFrozen(),
    engine.isFadWindow(),
    ...
    liquidationStalenessLimit,
    engine.fadMaxStaleness()
).maxStaleness;
```

It then passes the live liquidation limit for divergence check:

```
None
_resolveOraclePrice(
    pythUpdateData,
    1e8,
    maxStaleness,
    liquidationStalenessLimit
);
```

Inside `_computeBasketPrice()`, the individual feed staleness check uses the relaxed `maxStaleness`:

```
None
if (OracleFreshnessPolicyLib.isStale(uint64(p.publishTime),
    maxStaleness, block.timestamp)) {
    _revertOraclePriceTooStale();
}
```

But the basket divergence check uses the stricter limit (passed as `maxPublishTimeDivergence`):

```
None
if (maxPublishTime > minPublishTime + maxPublishTimeDivergence) {
    _revertOraclePriceTooStale();
}
```

This confirms that during oracle-frozen windows:

- individual feed staleness uses `fadMaxStaleness`
- basket timestamp difference still uses `liquidationStalenessLimit`

Concrete Path Flow

1. During an oracle-frozen window, individual feed freshness uses `fadMaxStaleness` (3 days).
2. The basket publish-time divergence limit remains `liquidationStalenessLimit` (15 seconds).
3. Two valid feeds diverge by 16 seconds.
4. The basket divergence check rejects the call because 16 seconds > 15 seconds.

Impact

Liquidations can be unavailable during oracle-frozen windows for affected [multi-feed baskets](#). If unsafe positions become liquidatable during that period, keepers may be unable to close them until feeds resume.

Recommended Fix

Use a frozen-window-aware divergence limit for liquidation baskets.

Response

Confirmed still fixed after oracle rewrite: [Plether-Fi/plether-core#15](#)

SC-M2: Valid Same-side Increases are Rejected as Dust Orders

Summary

`CfdEngine` rejects valid same-side position increases because the liquidation bounty check is incorrectly applied to the small incremental order notional instead of the resulting total live position notional, causing healthy positions to be flagged as `POSITION_TOO_SMALL`. This occurs because, while `OpenAccountingLib.buildOpenState()` correctly computes the resulting position size, it incorrectly stores the `state.notionalUsdc` value as only the notional

value of the incremental order size (`sizeDelta`). `CfdEnginePlanLib` then uses this incremental notional for the `POSITION_TOO_SMALL` check, resulting in the rejection of a small same-side increase, even when the final live position is valid and above the bounty floor.

Impact

Valid same-side increases can be rejected even when the final live position is healthy.

Recommended Fix

Apply the `minimum` liquidation-bounty check to the resulting live position notional, not the incremental order notional.

Response

`openState.notionalUsdc` is the incremental order notional and is still used for fees, but it was incorrectly reused for the live-position liquidation bounty floor. This pr fixes `planOpen` to check the resulting live position notional instead, so valid same-side increases are no longer rejected as `POSITION_TOO_SMALL` in [Plether-Fi/plether-core#28](#).

The related router/preview failure-policy handling was fixed in another/previous branch; since it routes through `planOpen` - [Plether-Fi/plether-core#24](#).

SC-M3: Fresh negative `vpiAccrued` is omitted from post-open IMR projection, allowing rebate-backed opens to settle below live initial margin

Summary

The `planOpen` function admits rebate-backed positions that settle immediately below the live Initial Margin Requirement (IMR). This occurs because during open admission, the planner credits rebates into projected collateral, but the function `_buildPostOpenRiskState(...)` fails to project the fresh negative `vpiAccrued` liability. While the planner approves the open using the rebate-enhanced collateral, the fresh VPI liability is written during live settlement and later subtracted from equity by risk accounting. This mismatch allows the position to be approved while ignoring the immediate VPI liability.

Example showing the resulting undercollateralization:

1. Alice opens a large BULL position to create skew.
2. Bob opens the opposite BEAR side with:
 - a. `sizeDelta = 300_000e18`
 - b. `marginDelta = 4000e6`
3. `previewOpenRevertCode(...)` returns OK.
4. `processOrderTyped(...)` succeeds at 1e8.
5. Immediately after execution, at the same mark:
 - a. the position exists,
 - b. `netEquityUsdc < initialMarginRequirementUsdc`,
 - c. `getWithdrawableUsdc(...) == 0`.

The residual planner/live mismatch remains for nonzero-margin rebate-backed opens.

Impact

This allows the engine to admit a position that is already below intended live **initial margin** at the moment it is opened, without any adverse price movement and undercollateralized positions can be admitted.

Recommended Fix

Keep the documented rebate-credit behavior, but make open admission project the same VPI state that live settlement writes.

Inside `_buildPostOpenRiskState(...)`, add:

```
projectedPosition.vpiAccrued = snap.position.vpiAccrued + delta.posVpiAccruedDelta;
```

Response

Fixed in [Plether-Fi/plether-core#22](#)

SC-M4: Risk Free Option in 60Sec Oracle Window

Summary

The `OrderRouter` contains a free option attack vector that allows a sophisticated attacker to extract risk-free profits from liquidity providers. By combining three protocol mechanisms , the 60-second oracle staleness window, the global FIFO queue's keeper-submitted oracle data, and a `SOLVENCY_EXCEEDED`→`RefundUser` margin-return path triggered by LP withdrawal , an attacker can effectively hold a costless directional option on the underlying price for up to one minute per order pair.

The attack is enabled by three core components:

- **Component 1: The 60-Second Oracle Window.** The `orderExecutionStalenessLimit` is set to 60 seconds. A keeper can select any Pyth price tick from this window, subject to liveness constraints.
- **Component 2: The Dual-Order Oracle Price Selection Exploit.** An attacker commits two sequenced orders (BULL/BEAR) at time T. As their own keeper, they execute the losing BEAR order (N) using an earlier price tick (T+5s/P₀) for minimal loss (fee only), and immediately execute the winning BULL order (N+1) using the later, optimal price tick (T+55s/P₀ × 1.03) for a profit, which is profitable if the price move clears the base fee of 0.08%.
- **Component 3: The LP Order Cancellation.** When a price move is too large, the attacker can trigger a full margin refund on the losing order by manipulating the vault depth. This is done by depositing to the junior tranche, then withdrawing the LP after determining the profitable direction. Executing the losing order then triggers `SOLVENCY_EXCEEDED` due to reduced vault depth. This propagates as `ProtocolStateInvalidated` → `RefundUser`, granting a full margin return.

The attacker recovers LP funds, the losing margin, and net profit from the winning side. The attack is constrained by the likelihood of a price move larger than 0.08% but offers risk-free profit for MEV searchers as soon as such an opportunity surfaces.

Impact

The attack is constrained by the likelihood of a price move larger than 0.08% and the exact oracle fees. However, as soon as such an opportunity surfaces, it is risk-free profit for [MEV searchers](#).

Recommended Fix

Configure fees in such a way that this attack can not be profitable.

Response

Fixed in [Plether-Fi/plether-core#24](#).

The following fix is large and requires a full audit.

SC-M5: Plain Stale Reconcile Intervals Back-Accrue Senior Yield.

Summary

[HousePool._reconcile\(\)](#) does not discard stale time for senior-yield accrual. The mechanism for this vulnerability is that [yieldElapsed](#) is computed using [lastSeniorYieldCheckpointTime](#) before mark freshness is checked, but [lastSeniorYieldCheckpointTime](#) is only updated if the mark is fresh. The sequence is as follows: A fresh reconcile at time t_0 sets the checkpoint. If the mark later becomes stale, a subsequent stale reconcile() skips yield calculations but does not advance the checkpoint. When the mark is later refreshed, the next fresh reconcile calculates yield elapsed from the old checkpoint at t_0 , thereby accruing senior yield across the entire stale interval, contradicting the rule that stale reconcile must not back-accrue senior yield.

Impact

Senior LPs can receive yield for stale intervals that should have been discarded. That creates a senior-favored accounting mismatch.

Recommended Fix

A straightforward fix is to checkpoint to the stale policy boundary, for example using the latest safe non-stale anchor such as $\max(\text{lastSeniorYieldCheckpointTime}, \text{lastMarkTime})$, rather than leaving the old checkpoint untouched.

Response

Didn't address this issue directly but it no longer exists after senior rewrite (with no stale-time senior-yield debt) in [Plether-Fi/plether-core#23](#).

The following fix is large and requires a full audit.

SC-M6: If Price Cap is Reached Protocol Becomes Unusable

Summary

The `CAP_PRICE` in the perpetuals protocol is set as an `immutable` variable at construction time with no mechanism to update it. The protocol is designed around a specific synthetic asset, where `CAP_PRICE = 2e8` (\$2.00). If the real-world price of the underlying asset ever reaches or exceeds this hardcoded ceiling, the protocol becomes permanently non-functional: all oracle prices are clamped to `CAP_PRICE`, mark price updates stall, and no new positions can be meaningfully opened or closed. `CAP_PRICE` is stored as a Solidity `immutable`, meaning it is baked into the contract bytecode at deployment and can never be changed. The deployment scripts hardcode it to `2e8`: **DeployPerpsArbitrumSepolia - Lines: 21-22**. During order execution, every oracle price is silently clamped to `CAP_PRICE` before being forwarded to the engine: **OrderOracleExecution - Lines: 129-133**. The engine itself enforces the same clamp on every mark price update: **CfdEngine - Lines: 484**. Additionally, the commit-reference price used for order pre-filtering is also clamped: **OrderOracleExecution - Lines: 298-300**. Because `CAP_PRICE` affects PnL calculation, margin solvency checks, liquidation thresholds, and the mark price cache, the entire trade lifecycle is dependent on the oracle price being below this ceiling.

Impact

If the price cap is reached, the oracle and mark prices are permanently frozen, which breaks the payoff structure for BULL position holders who cannot realize gains above the cap. Furthermore, BEAR liquidations are flawed because positions are evaluated at the capped price, potentially allowing insolvent positions to survive. This issue is permanent as there is **no admin path to fix this**; changing the immutable `CAP_PRICE` requires deploying a new contract and migrating all state.

Recommended Fix

Replace the `immutable CAP_PRICE` with a mutable, governance-controlled state variable and add a setter restricted to the owner or a timelock.

Response

Acknowledged

SC-M7: Order Router can be DOSed by creating worthless withdraw orders

Summary

The `OrderRouter` FIFO queue can be grieved by an attacker who floods it with cheap close orders, forcing keepers to waste gas executing worthless entries while paying more to execute than it costs to commit. Close orders require a flat `closeOrderExecutionBountyUsdc` fee and no minimum `sizeDelta` greater than 0. An attacker can submit a stream of dust-sized close orders (e.g., `sizeDelta = 1 wei`), each of which:

1. Passes all validation checks.
2. Occupies a slot in the global FIFO queue.
3. Blocks subsequent legitimate orders.

Rational keepers refuse to process these orders because the gas cost to execute (including Pyth update) exceeds the execution bounty. Since `maxPendingOrders` is only enforced per account, the attacker can use multiple wallets to saturate the global queue, causing queue stall and keeper abandonment.

Impact

An attacker can execute a Denial of Service (DoS) by saturating the OrderRouter's FIFO queue with dust-sized, worthless close orders. This causes legitimate orders to be blocked, leading to queue stall, keeper abandonment, and an inability to process critical transactions.

Recommended Fix

Enforce a minimum `sizeDelta` for close orders, proportional to a meaningful notional value (e.g. the same floor used for open orders). Concretely, before the `executionBountyUsdc = closeOrderExecutionBountyUsdc` assignment, add a check:

```
if (sizeDelta < minCloseSize) revert ...
```

where `minCloseSize` is a configurable parameter (set via `applyRouterConfig`) representing the smallest economically meaningful close quantity, for example, the `sizeDelta` whose notional at the current commit price equals `minOpenOrderExecutionBountyUsdc`. This ensures the keeper's bounty always covers execution costs and eliminates the incentive to flood the queue with dust closes.

Response

Fixed in [Plether-Fi/plether-core#27](#)

Patch now rejects partial closes below the engine's minimum meaningful notional floor, while still allowing full closes of tiny residual positions so users don't get stuck.

SC-L1: Permissionless zero-value senior withdraw can persist overstated HousePool tranche principal

Summary

The HousePool's tranche principal can be materially overstated on-chain due to a flaw in `_reconcile()` where pending claimant buckets are applied without MTM/liability adjustments. A zero-amount withdrawal can prevent the state from self-correcting, leaving raw `seniorPrincipal` / `juniorPrincipal` and `getVaultLiquidityView()` falsely high. The principal is overstated when `_reconcile()` applies pending claimant buckets without capping the credit against the liability-adjusted distributable amount. This allows `_applyPendingClaimantBucketsLive()` to temporarily set raw `seniorPrincipal`/`juniorPrincipal` above the true NAV. While subsequent vault actions normally correct this false state, a zero-amount withdrawal via `TrancheVault.withdraw(0, receiver, owner)` triggers the flawed reconcile. Since `HousePool.withdrawSenior(0)` returns immediately without a final reconcile, the overstated principal persists until another action is taken. For instance, a PoC showed the raw principal at 100,000 USDC, while the correct value was 49,960 USDC.

Impact

Any address can cheaply persist materially false HousePool tranche principal/NAV after a pending claimant bucket is present.

This can mislead frontends using `getVaultLiquidityView()`.

Recommended Fix

Make pending claimant bucket settlement part of the same liability-adjusted reconcile kernel instead of applying it afterward as an uncapped principal credit.

Response

Fixed in [Plether-Fi/plether-core#29](#). Pending recap/revenue is now capped to what's actually settleable after MTM/deferred liabilities. Anything that can't be settled yet stays pending, stays reserved everywhere, and gets applied later once it's backed.

Also: the zero-withdraw trigger was already blocked on [Plether-Fi/plether-core#23](#), but the accounting issue underneath was still real, so I fixed the root cause.

SC-I1: Remove Dead Code from HousePool

Summary

`HousePool` contains two private helper functions, `_distributeRevenue` and `_absorbLoss`, that are never called anywhere in the contract. Both are marked with `@audit dead code` comments. They duplicate logic already performed internally by `_reconcile` via `HousePoolWaterfallAccountingLib`, and their presence misleads readers into thinking there is a direct revenue/loss dispatch path that does not actually exist.

Code Snippet

```
None

function _distributeRevenue() //@audit dead code

function _absorbLoss() //@audit dead code
```

Additionally, the `Reconciled` event is only emitted from these two dead functions, meaning the event itself is also effectively dead and never fires in live execution.

Recommended Fix

Delete `_distributeRevenue`, `_absorbLoss`, and the `Reconciled` event. All revenue distribution and loss absorption is performed through `_reconcile` → `HousePoolReconcilePlanLib.planReconcile`, which is the canonical path. If a reconcile event

is genuinely desired for observability, it should be emitted directly inside `_reconcile` after the waterfall state is updated, not via a separate dead-code wrapper.

Response

Done in [Plether-Fi/plether-core#19](#)

SC-I2: Missing Order Cancellation Mechanism

Summary

The `OrderRouter` contract provides no way for users to cancel pending orders, permanently locking committed margin and execution bounties in the queue until a keeper executes or the order expires.

Vulnerability Detail

Orders committed via `commitOrder` reserve margin via `_reserveCommittedMargin` and an execution bounty via `_reserveExecutionBounty`. The only exit paths are:

1. Keeper execution via `executeOrder` / `executeOrderBatch`
2. Passive expiry (skipped during execution when `block.timestamp - order.commitTime > maxOrderAge`)

There is no user-callable cancellation function. If keepers go offline, are censoring a user, or if `maxOrderAge` is set to 0 (which disables expiry, as checked in `executeOrderBatch`), committed funds become indefinitely inaccessible. A user who commits an order with incorrect parameters (wrong `targetPrice`, unintended `sizeDelta`) has no recourse.

Expiry is only processed during keeper execution, not on demand:

```
None
// executeOrderBatch expiry only runs inside the keeper loop
if (maxOrderAge > 0 && block.timestamp - order.commitTime >
maxOrderAge) {
    ...
    _cleanupOrder(...);
}
```

`maxOrderAge` can be set to 0, disabling expiry entirely, `applyRouterConfig` does not enforce lower bound: `maxOrderAge = config.maxOrderAge;`

Recommended Fix

Add a `cancelOrder(uint64 orderId)` or acknowledge this finding.

Response

Acknowledged